/ A 1x1/1/XX Convolutional Layer:

we can use a 1x1/1/XX convolutional layer to increase or decrease the num of output channels (=XX) in this convolutional layer,
because the num of output channels in a convolutional layer can be customized as a hyperparameter.
that is, the num of input channels in the next convolutional layer will increase or decrease correspondingly, too.
in this case, we can efficiently control the num of input channels, the num of weights, and the num of floating point operations in the next convolutional layer at the micro level.
furthermore, we can also control the overall num of channels, the overall num of weights, and the overall num of floating point operations in a module or even a neuron network at the macro level.

2. GoogleNet:

A. a typical inception module in a GoogleNet has several characteristics:

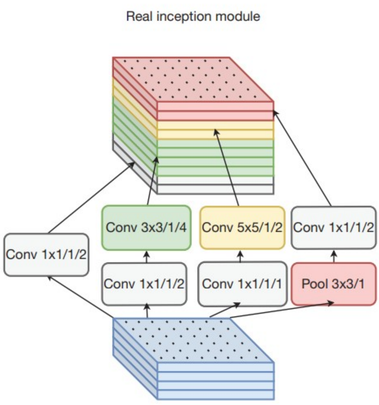
⇒ the parallel execution structure:
the input data is simultaneously distributed across multiple parallel chains of convolutional layers that compute the input data side by side at the same hierarchical level
rather than flowing purely through a single sequential chain of convolutional layers.

⇒ multi-scale features extraction:
the module establishes distinct, independent convolutional layers with different kernel sizes (Ex: 3x3, 5x5...).
each different size kernel independently extracts feature maps with a different size receptive field,
allowing the neuron network to recognize the same feature with various physical scales in an image simultaneously.

⇒ channels concatenation:
all parallel chains of convolutional layers in the module maintain identical width and height dimensions of feature maps
by using a stride of 1 and an appropriate padding mechanism.
in this case, all resulting output data from each chain of convolutional layers can be concatenated along to become a single set of cohesive output channels,
which is passed to the following inception module or the following fully connected layer.

⇒ the benefit of applying a 1x1/1/XX convolutional layer:
to maintain a lower overall weights count in an inception module, an inception module can apply a 1x1/1/XX convolutional layer before a 3x3/1/XX convolutional layer or a 5x5/1/XX convolutional layer.
this reduces the num of input channels fed into a 3x3/1/XX convolutional layer or a 5x5/1/XX convolutional layer, effectively reducing the weights count of a 3x3/1/XX convolutional layer or a 5x5/1/XX convolutional layer.
additionally, a 1x1/1/XX convolutional layer is applied immediately after a 3x3 max pooling operation.
this allows for complete control over the total num of channels for an inception module, thereby tightly controlling the overall weights count for an inception module.

⇒



B. auxiliary classifiers:

⇒ to enable training of a deeper GoogleNet, we add auxiliary classifiers at different points in a GoogleNet.
an auxiliary classifier is similar to what we typically would put at the top of a CNN.
that is, it is a fully connected layer with SoftMax as activation functions.

⇒ the intent of auxiliary classifiers is to be able to inject gradient descent at these intermediate points, avoid the Vanishing Gradient problem,
and thereby ensure that big enough gradient descent values are propagated back to the first few convolutional layers with the Backpass process.
big enough gradient descent values augmented by auxiliary classifiers also enable the initial convolutional layers in a GoogleNet to be trained to behave much like they would behave in a shallower model (= a model with fewer layers).

⇒ auxiliary classifiers only exist during the training stage,
these side branches are completely discarded and only the primary backbone is left for the forwardpass process during the inference stage.

⇒

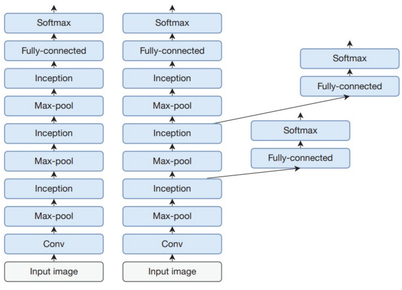


Figure 8-3 Left: Baseline network based on Inception modules. Right: The same network augmented with auxiliary classifiers.

C. a GoogleNet demonstrate that it is possible to make use of a more elaborate network architecture to build a deep, high-performing neuron network has a relatively smaller num of weights.

D.

Table 8-2 GoogLeNet Architecture

Layer Type	Details				Output size
Input	RGB image				224×224×3
Conv	7×7/2/64				112×112×64
Max pool	3×3/2				56×56×64
Conv	1×1/1/64				56×56×64
Conv	3×3/1/192				56×56×192
Max pool	3×3/2				28×28×192
Inception	1×1/1/64	1×1/1/96	1×1/1/16	3×3/1 pool	28×28×256
		3×3/1/128	5×5/1/32	1×1/1/32	
	Channel concatenation				
Inception	1×1/1/128	1×1/1/128	1×1/1/32	3×3/1 pool	28×28×480
		3×3/1/192	5×5/1/96	1×1/1/64	
	Channel concatenation				
Max pool	3×3/2				14×14×280
Inception	1×1/1/192	1×1/1/96	1×1/1/16	3×3/1 pool	14×14×512
		3×3/1/208	5×5/1/48	1×1/1/64	
	Channel concatenation				
Inception	1×1/1/160	1×1/1/112	1×1/1/24	3×3/1 pool	14×14×512
		3×3/1/224	5×5/1/64	1×1/1/64	
	Channel concatenation				
Inception	1×1/1/128	1×1/1/128	1×1/1/24	3×3/1 pool	14×14×512
		3×3/1/256	5×5/1/64	1×1/1/64	
	Channel concatenation				
Inception	1×1/1/112	1×1/1/144	1×1/1/32	3×3/1 pool	14×14×512
		3×3/1/288	5×5/1/64	1×1/1/64	
	Channel concatenation				
Inception	1×1/1/256	1×1/1/160	1×1/1/32	3×3/1 pool	14×14×832
		3×3/1/320	5×5/1/128	1×1/1/128	
	Channel concatenation				
Max pool	3×3/2				7×7/832
Inception	1×1/1/256	1×1/1/160	1×1/1/32	3×3/1 pool	7×7×832
		3×3/1/320	5×5/1/128	1×1/1/128	
	Channel concatenation				
Inception	1×1/1/384	1×1/1/192	1×1/1/48	3×3/1 pool	7×7×1,024
		3×3/1/384	5×5/1/128	1×1/1/128	
	Channel concatenation				
Avg pool	7×7/1				1,024
Dropout	40%				1,024
FC (softmax)	1,000				1,000